Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

Aim

To analyze the Software Development Life Cycle (SDLC) models used in the development of a real-world software application and identify the most suitable SDLC model.

Objective

- Study different SDLC models.
- Analyze the development process of a real-world software application.
- Compare the advantages and limitations of various SDLC models.
- Recommend the most appropriate SDLC model for the application.

Selected Software Application

Online Ticket Booking System (e.g., BookMyShow/IRCTC/RedBus)

Overview

An Online Ticket Booking System lets customers search for movies, buses, or trains, view real-time seat availability, and complete a booking end-to-end without visiting a counter. Key capabilities include:

- Register/Login
- Search movies, buses, or trains by route, date, or location
- View seat layout and select seats
- Book tickets and receive instant confirmation
- Make online payments through multiple payment modes
- Cancel or reschedule bookings with refund tracking
- Provide ratings and reviews for the service

Functional Requirements

Customer Module

- User Registration
- Login
- Search Movies/Buses/Trains
- Select Seats
- Book Ticket
- Online Payment
- Cancel/Reschedule Booking

Service Provider Module

- Manage Schedules
- Manage Seat Availability
- Accept/Reject Booking Requests
- Update Booking Status

Notification Module

- Booking Confirmation
- Payment Confirmation
- Cancellation Alerts

Admin Module

- User Management
- Service Provider Management
- Reports
- Analytics

SDLC Models Analysis

Each SDLC model was evaluated against the specific demands of a ticket booking platform, such as handling real-time seat availability, frequent schedule changes, and sudden traffic surges during festive or holiday seasons.

SDLC Model	Applicability	Key Reason
Waterfall	Not Suitable	Rigid, sequential structure cannot adapt to frequently changing show timings, fares, or schedules.
Prototype	Partially Suitable	Helpful for validating the seat-selection and payment UI early, but too slow for full-scale delivery.
Spiral	Suitable	Strong risk analysis fits well for critical payment and security features, but adds cost and time.
Agile	Highly Suitable	Short sprints allow quick rollout of new routes, offers, and fare changes based on customer feedback.
DevOps	Highly Suitable	Continuous deployment keeps the system stable and scalable during high-traffic booking rushes.

Why Agile (with DevOps) is Best?

Reasons specific to a ticket booking system

- Show timings, bus/train schedules, and fares change frequently and need fast updates.
- Seat availability must be reflected in real time to avoid double-booking.
- Demand spikes sharply during holidays and festive seasons, requiring quick scaling.
- Payment failures and booking bugs need to be fixed and released without delay.
- Ongoing customer feedback helps refine the search, seat-selection, and checkout flow.

Agile Development Process

Requirement Gathering
↓
Sprint Planning
↓
Design
↓
Development
↓
Testing
↓
Deployment
↓
Customer Feedback
↓
Next Sprint

SDLC Phases

1. Requirement Analysis

- Customer requirements — search, seat selection, secure checkout
- Service provider requirements — schedule and inventory management
- Payment Gateway integration for cards, UPI, and wallets
- Real-time Seat Availability and locking system

Deliverables

- Software Requirement Specification (SRS)

2. System Design

Design Includes

- Database Design
- UI Design
- Architecture Design
- API Design

Deliverables

- UML Diagrams
- Database Schema

3. Development

Modules Developed:

- Login Module
- Booking Module
- Payment Module

- Notification Module

Programming Languages:

- Java/Python
- React
- Node.js

Database:

- MySQL / PostgreSQL

4. Testing

Testing Types:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

Example Test Case

Test Case	**Expected Result**
Login	Successful Login
Invalid Password	Error Message
Book Ticket	Booking Created
Payment	Payment Successful

5. Deployment

Deployment Options

- Cloud Deployment
- Mobile App Release
- Web Server Deployment

Cloud Platforms

- AWS
- Azure
- Google Cloud

6. Maintenance

Activities

- Bug Fixes
- Security Updates
- Feature Enhancements
- Performance Optimization

Comparison of SDLC Models

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Simple and easy to manage	Difficult to accommodate changing requirements
Prototype	Medium	Better understanding of user requirements	Increased development cost
Spiral	High	Good risk management	Expensive and complex
Agile	Very High	Continuous improvement, customer feedback	Requires experienced team
DevOps	Very High	Continuous Integration and Deployment	Infrastructure complexity

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement Changes	Poor	Good	Excellent	Excellent	Excellent
Customer Feedback	Low	High	High	Very High	Very High
Risk Handling	Low	Medium	High	Medium	High
Delivery Speed	Slow	Medium	Medium	Fast	Very Fast
Cost	Low	Medium	High	Medium	Medium
Suitable for Ticket Booking App	No	Partially	Yes	Best	Best

Results

- The Online Ticket Booking System requires continuous feature updates and customer feedback.
- Agile SDLC supports iterative development, quick releases, and flexibility.
- DevOps complements Agile by enabling continuous integration, testing, and deployment.
- Therefore, Agile with DevOps practices is the most suitable approach for developing and maintaining an online ticket booking application.

Conclusion

The case study demonstrates that selecting the appropriate SDLC model depends on project requirements. For dynamic applications such as online ticket booking systems, Agile provides flexibility, faster delivery, and continuous customer involvement, while DevOps enhances deployment automation and operational efficiency. This combination ensures high software quality, rapid feature delivery, and improved user satisfaction.